

Модуль 1 Практика 3

Выполнила: Галашина А.В.

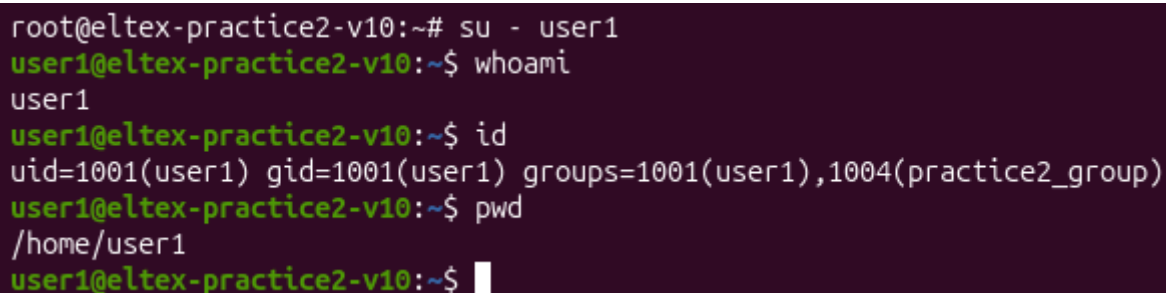
РАЗДЕЛ 1

Подключимся к серверу через ssh и создадим отдельную сессию для практики 3

screen -S practice3

(функция run в отчете используется для логирования)

1 Войдите под пользователем user1 из практики 2 (su - user1)



```
root@eltex-practice2-v10:~# su - user1
user1@eltex-practice2-v10:~$ whoami
user1
user1@eltex-practice2-v10:~$ id
uid=1001(user1) gid=1001(user1) groups=1001(user1),1004(practice2_group)
user1@eltex-practice2-v10:~$ pwd
/home/user1
user1@eltex-practice2-v10:~$
```

Рис. 1 – Результат

2 Подсчитайте количество процессов, имеющих несколько потоков выполнения

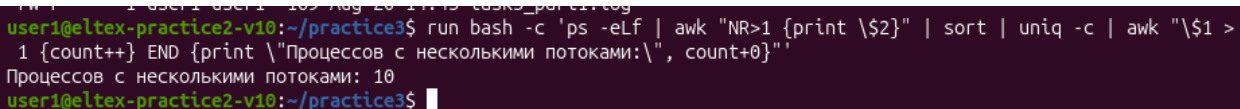
run bash -c 'ps -eLf | awk "NR>1 {print \\$2}" | sort | uniq -c | awk "\\$1 > 1 {count++} END {print \"Процессов с несколькими потоками:\", count+0}"'

ps -eLf — список всех процессов с потоками (флаг -L);

awk "NR>1 {print \\$2}" — пропускаем заголовок, берём PID каждого потока;

sort | uniq -c — группируем по PID, считаем число потоков;

awk "\\$1 > 1" — оставляем процессы, у которых потоков больше 1.



```
user1@eltex-practice2-v10:~/practice3$ run bash -c 'ps -eLf | awk "NR>1 {print \$2}" | sort | uniq -c | awk "\$1 > 1 {count++} END {print \"Процессов с несколькими потоками:\", count+0}"'
Процессов с несколькими потоками: 10
user1@eltex-practice2-v10:~/practice3$
```

Рис. 2 – Результат

3 Запустите top и настройте вывод полей с информацией о процессе

следующим образом:

- удалите поля VIRT, RES, SHR;
- добавьте поле RUSER и сделайте так, чтобы это поле было показано после поля USER;

Внутри top:

Нажмем f - откроется Fields Manager;

Стрелками найдите VIRT, RES, SHR — на каждом нажмите пробел (снять *);

Найдем RUSER — нажмем пробел (поставить *);

на строке RUSER нажмем Alt+l — поле «берётся для перемещения»;

жмем ↑ (или Alt+k), пока RUSER не встанет строкой сразу под USER;

нажмем Enter (или ← / Alt+h) — зафиксировать позицию.

Нажмите q — вернуться к таблице.

Нажмите q чтобы выйти из top.

```

eltex-v10@eltex-2026-summer: ~ — ssh eltex-v10@217.71.138.1 -p 44556
Fields Management for window 1:Def, whose current sort field is %CPU
Navigate with Up/Dn, Right selects for move then <Enter> or Left commits,
'd' or <Space> toggles display, 's' sets sort. Use 'q' or <Esc> to end!

* PID      = Process Id          LXC      = LXC container name
* USER     = Effective User Name RSan     = RES Anonymous (KiB)
* PR       = Priority          RSfd     = RES File-based (KiB)
* NI       = Nice Value       RSlk     = RES Locked (KiB)
VIRT      = Virtual Image (KiB) RSsh     = RES Shared (KiB)
RES       = Resident Size (KiB) CGNAME    = Control Group name
SHR       = Shared Memory (KiB) NU        = Last Used NUMA node
* S        = Process Status   LOGID    = Login User Id
* %CPU     = CPU Usage        EXE       = Executable Path
* %MEM     = Memory Usage (RES) RSS       = Res Mem (smaps), KiB
* TIME+    = CPU Time, hundredths PSS      = Proportion RSS, KiB
* COMMAND  = Command Name/Line PSan     = Proportion Anon, KiB
PPID      = Parent Process pid PSfd     = Proportion File, KiB
UID       = Effective User Id  PSsh     = Proportion Shrd, KiB
RUID      = Real User Id      USS      = Unique RSS, KiB
* RUSER    = Real User Name   ioR      = I/O Bytes Read
SUID      = Saved User Id     ioRop    = I/O Read Operations
SUSER     = Saved User Name   ioW      = I/O Bytes Written
GID       = Group Id         ioWop    = I/O Write Operations
GROUP     = Group Name       AGID     = Autogroup Identifier
PGRP      = Process Group Id AGNI     = Autogroup Nice Value
TTY       = Controlling Tty   STARTED  = Start Time from boot
TPGID     = Tty Process Grp Id ELAPSED  = Elapsed Running Time
SID       = Session Id       %CUU     = CPU Utilization
nTH       = Number of Threads %CUC     = Utilization + child
P         = Last Used Cpu (SMP) nsCGROUP = CGRP namespace Inode
TIME      = CPU Time        nsTIME   = TIME namespace Inode
SWAP      = Swapped Size (KiB)
CODE      = Code Size (KiB)
DATA      = Data+Stack (KiB)
nMaj      = Major Page Faults
nMin      = Minor Page Faults
nDRT      = Dirty Pages Count
WCHAN     = Sleeping in Function
Flags     = Task Flags <sched.h>
CGROUPS   = Control Groups
SUPGIDS   = Supp Groups IDs
SUPGRPS   = Supp Groups Names
TGID      = Thread Group Id
OOMa      = OOMEM Adjustment
OOMs      = OOMEM Score current
ENVIRON   = Environment vars
vMj       = Major Faults delta
vMn       = Minor Faults delta
USED      = Res+Swap Size (KiB)
nsIPC     = IPC namespace Inode
nsMNT     = MNT namespace Inode
nsNET     = NET namespace Inode
nsPID     = PID namespace Inode
nsUSER    = USER namespace Inode
nsUTS     = UTS namespace Inode

```

Рис. 3 – Результат

PID	USER	RUSER	PR	NI	S	%CPU	%MEM	TIME+	COMMAND
4	root	root	0	-20	I	0.0	0.0	0:00.00	kworker/R-rcu_g
5	root	root	0	-20	I	0.0	0.0	0:00.00	kworker/R-rcu_p
6	root	root	0	-20	I	0.0	0.0	0:00.00	kworker/R-slub_
7	root	root	0	-20	I	0.0	0.0	0:00.00	kworker/R-netns
10	root	root	0	-20	I	0.0	0.0	0:00.00	kworker/0:0H-events_highpri
12	root	root	0	-20	I	0.0	0.0	0:00.00	kworker/R-mm_pe
13	root	root	20	0	I	0.0	0.0	0:00.00	rcu_tasks_kthread
14	root	root	20	0	I	0.0	0.0	0:00.00	rcu_tasks_rude_kthread
15	root	root	20	0	I	0.0	0.0	0:00.00	rcu_tasks_trace_kthread
16	root	root	20	0	S	0.0	0.0	0:00.17	ksoftirqd/0
17	root	root	20	0	I	0.0	0.0	0:05.90	rcu_preempt
18	root	root	rt	0	S	0.0	0.0	0:03.54	migration/0
19	root	root	-51	0	S	0.0	0.0	0:00.00	idle_inject/0
20	root	root	20	0	S	0.0	0.0	0:00.00	cpuhp/0
21	root	root	20	0	S	0.0	0.0	0:00.00	cpuhp/1
22	root	root	-51	0	S	0.0	0.0	0:00.00	idle_inject/1
23	root	root	rt	0	S	0.0	0.0	0:03.56	migration/1
24	root	root	20	0	S	0.0	0.0	0:00.13	ksoftirqd/1
26	root	root	0	-20	I	0.0	0.0	0:00.00	kworker/1:0H-events_highpri
27	root	root	20	0	S	0.0	0.0	0:00.00	kdevtmpfs
28	root	root	0	-20	I	0.0	0.0	0:00.00	kworker/R-inet_
29	root	root	20	0	S	0.0	0.0	0:00.03	kauditd
30	root	root	20	0	S	0.0	0.0	0:00.20	khungtaskd
32	root	root	20	0	S	0.0	0.0	0:00.00	oom_reaper
34	root	root	0	-20	I	0.0	0.0	0:00.00	kworker/R-write
35	root	root	20	0	S	0.0	0.0	0:19.01	kcompactd0
36	root	root	25	5	S	0.0	0.0	0:00.00	ksmd
38	root	root	39	19	S	0.0	0.0	0:00.00	khugepaged
39	root	root	0	-20	I	0.0	0.0	0:00.00	kworker/R-kinte
40	root	root	0	-20	I	0.0	0.0	0:00.00	kworker/R-kbloc
41	root	root	0	-20	I	0.0	0.0	0:00.00	kworker/R-blkcg
42	root	root	-51	0	S	0.0	0.0	0:00.00	irq/9-acpi
43	root	root	0	-20	I	0.0	0.0	0:00.00	kworker/R-tpm_d
44	root	root	0	-20	I	0.0	0.0	0:00.00	kworker/R-ata_s
45	root	root	0	-20	I	0.0	0.0	0:00.00	kworker/R-md
46	root	root	0	-20	I	0.0	0.0	0:00.00	kworker/R-md_bi
47	root	root	0	-20	I	0.0	0.0	0:00.00	kworker/R-edac-
48	root	root	0	-20	I	0.0	0.0	0:00.00	kworker/R-devfr
49	root	root	-51	0	S	0.0	0.0	0:00.00	watchdogd
50	root	root	0	-20	I	0.0	0.0	0:00.00	kworker/R-quota
51	root	root	0	-20	I	0.0	0.0	0:01.29	kworker/1:1H-kblockd
52	root	root	20	0	S	0.0	0.0	0:00.00	kswapd0
53	root	root	20	0	S	0.0	0.0	0:00.00	ecryptfs-kthread
54	root	root	0	-20	I	0.0	0.0	0:00.00	kworker/R-kthro
55	root	root	0	-20	I	0.0	0.0	0:00.00	kworker/R-acpi_
57	root	root	20	0	S	0.0	0.0	0:00.00	scsi_eh_0
58	root	root	0	-20	I	0.0	0.0	0:00.00	kworker/R-scsi_
59	root	root	20	0	S	0.0	0.0	0:00.00	scsi_eh_1

Рис. 4 – Результат

4 В другом терминальном окне выполните команду `passwd` и оставьте ее в состоянии запроса текущего пароля

```
root@eltex-practice2-v10:~# su - user1
user1@eltex-practice2-v10:~$ cd practice3/
user1@eltex-practice2-v10:~/practice3$ passwd
Changing password for user1.
Current password: 
```

Рис. 5 – Результат

5 Перейдите в терминальное окно с `top` и выполните следующие действия:

- выведите все процессы, для которых реальным пользователем является пользователь, которым вы вошли в сеанс;

- найдите процесс, запущенный командой passwd;

PID	USER	RUSER	PR	NI	S	%CPU	%MEM	TIME+	COMMAND
31852	user1	user1	20	0	S	0.0	0.1	0:00.03	bash
39525	user1	user1	20	0	S	0.0	0.1	0:00.01	bash
39535	root	user1	20	0	S	0.0	0.1	0:00.00	passwd

Рис. 6 – Результат

- отправьте этому процессу сигналы 15 (SIGTERM), 2 (SIGINT), 3 (SIGQUIT), 9(SIGKILL)

PID to signal/kill [default pid = 31852] 39535									
PID	USER	RUSER	PR	NI	S	%CPU	%MEM	TIME+	COMMAND
31852	user1	user1	20	0	S	0.0	0.1	0:00.03	bash
39455	user1	user1	20	0	R	0.0	0.2	0:00.18	top
39525	user1	user1	20	0	S	0.0	0.1	0:00.01	bash
39535	root	user1	20	0	S	0.0	0.1	0:00.00	passwd

Рис. 7 – Результат

Send pid 39535 signal [15/sigterm]									
PID	USER	RUSER	PR	NI	S	%CPU	%MEM	TIME+	COMMAND
31852	user1	user1	20	0	S	0.0	0.1	0:00.03	bash
39455	user1	user1	20	0	R	0.0	0.2	0:00.18	top
39525	user1	user1	20	0	S	0.0	0.1	0:00.01	bash
39535	root	user1	20	0	S	0.0	0.1	0:00.00	passwd

Рис. 8 – Результат sigterm

Tasks: 123 total, 1 running, 122 sleeping, 0 stopped, 0 zombie									
%Cpu(s): 0.0 us, 0.2 sy, 0.0 ni, 99.8 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st									
MiB Mem : 3915.9 total, 2122.4 free, 514.4 used, 1572.5 buff/cache									
MiB Swap: 3185.0 total, 3185.0 free, 0.0 used. 3401.4 avail Mem									
Send pid 39542 signal [15/sigterm] 2									
PID	USER	RUSER	PR	NI	S	%CPU	%MEM	TIME+	COMMAND
39525	user1	user1	20	0	S	0.0	0.1	0:00.01	bash
39542	root	user1	20	0	S	0.0	0.1	0:00.00	passwd

Рис. 9 – Результат sigint

```
Tasks: 123 total, 1 running, 122 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.0 us, 0.0 sy, 0.0 ni, 100.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 3915.9 total, 2114.4 free, 522.5 used, 1572.5 buff/cache
MiB Swap: 3185.0 total, 3185.0 free, 0.0 used. 3393.4 avail Mem
Send pid 39549 signal [15/sigterm] 3
```

PID	USER	RUSER	PR	NI	S	%CPU	%MEM	TIME+	COMMAND
39525	user1	user1	20	0	S	0.0	0.1	0:00.01	bash
39549	root	user1	20	0	S	0.0	0.1	0:00.00	passwd

Рис. 10 – Результат sigout

```
Tasks: 123 total, 1 running, 122 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.0 us, 0.0 sy, 0.0 ni, 99.8 id, 0.2 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 3915.9 total, 2114.1 free, 522.7 used, 1572.5 buff/cache
MiB Swap: 3185.0 total, 3185.0 free, 0.0 used. 3393.2 avail Mem
Send pid 39549 signal [15/sigterm] 9
```

PID	USER	RUSER	PR	NI	S	%CPU	%MEM	TIME+	COMMAND
39525	user1	user1	20	0	S	0.0	0.1	0:00.01	bash
39549	root	user1	20	0	S	0.0	0.1	0:00.00	passwd

Рис. 11 – Результат sigkill

```
passwd: password unchanged
user1@eltex-practice2-v10:~$ passwd
Changing password for user1.
Current password: Killed
user1@eltex-practice2-v10:~$
```

Рис. 12 – Результат sigkill

6 Выполните команду `vim ~/file_task3.txt` и нажмите Ctrl-Z

[illegible]

Рис. 15 – Результат

9 Измените число NICE у задания (sleep 600), сделав его равным 10

10 Проверьте, что число NICE у этого задания изменилось

```
user1@eltex-practice2-v10:~/practice3$ pgrep -f "sleep 600"
39639
user1@eltex-practice2-v10:~/practice3$ run renice -n 10 -p 39639
39639 (process ID) old priority 0, new priority 10
user1@eltex-practice2-v10:~/practice3$ run ps -o pid,ni,comm -p 39639
  PID  NI  COMMAND
 39639  10  sleep
user1@eltex-practice2-v10:~/practice3$
```

Рис. 16 – Результат

11 Сделайте задание vim ~/file task3.txt активным и выйдите из редактора

fg

:q

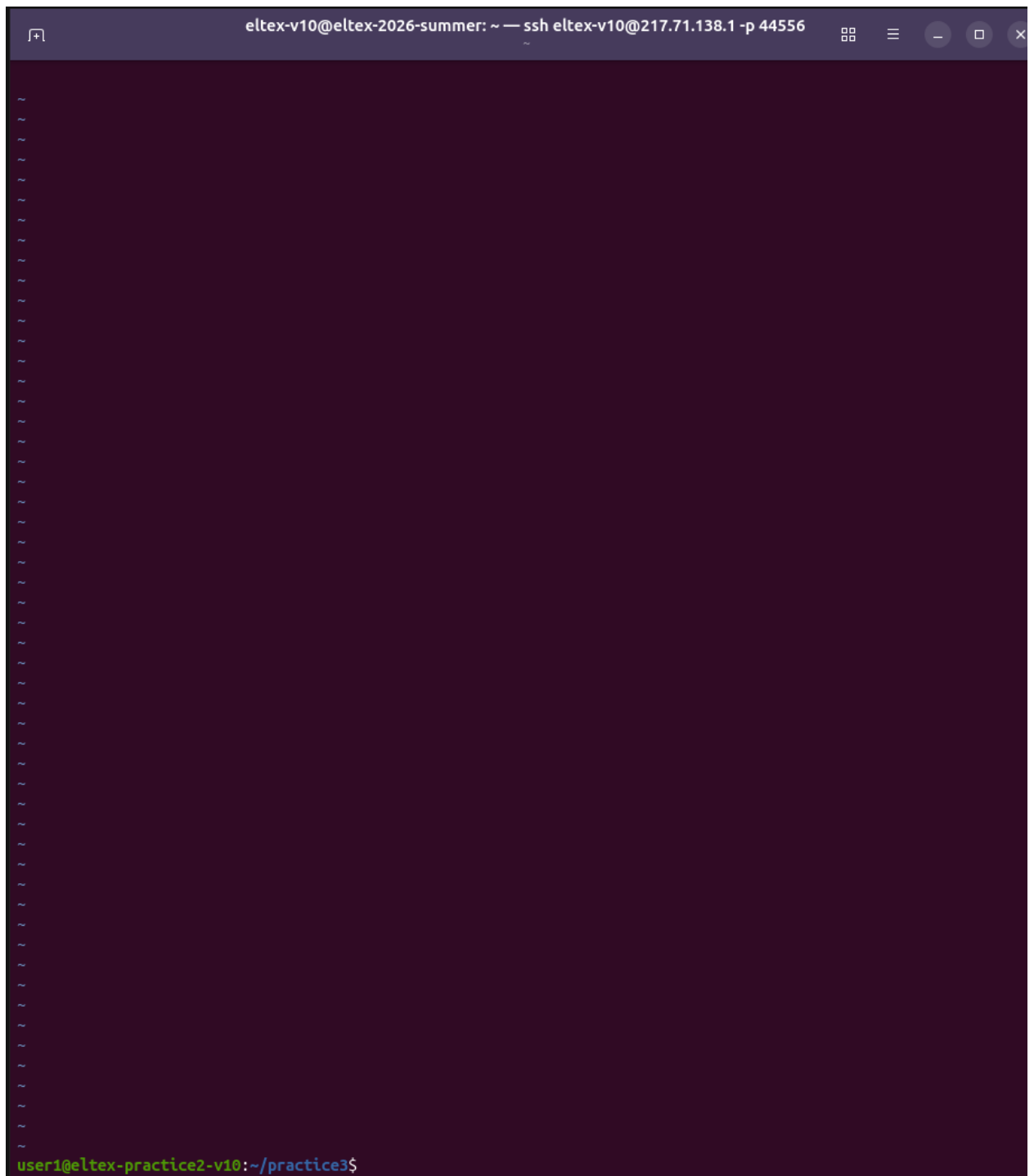


Рис. 17 – Результат

12 Отправьте сигнал 15 (SIGTERM) заданию sleep 600 и выполните команду jobs


```
user1@eltex-practice2-v10:~/practice3$ trap 'echo "Меня голыми руками не возьмёшь!"' SIGINT SIGQUIT
user1@eltex-practice2-v10:~/practice3$ echo "PID текущего shell: $$"
PID текущего shell: 31852
user1@eltex-practice2-v10:~/practice3$ kill -SIGINT $$
Меня голыми руками не возьмёшь!
user1@eltex-practice2-v10:~/practice3$ kill -SIGQUIT $$
Меня голыми руками не возьмёшь!
user1@eltex-practice2-v10:~/practice3$ trap - SIGINT SIGQUIT
user1@eltex-practice2-v10:~/practice3$
```

Рис. 19 – Результат

РАЗДЕЛ 2

1 Создайте скрипт на языке bash с именем cuckoo.sh, делающий следующее:

- Создает именованный канал с именем cuckoo.pid в каталоге /tmp/run/
- Записывает в файл cuckoo.log сообщение в формате: ДАТА ВРЕМЯ Startup!
- В процессе ожидания читает сообщения из именованного канала, на запрос «NAME[PID]: how much time do I have?» возвращает число N в диапазоне от 2 до 10, где NAME – имя обратившегося процесса и PID – его идентификатор, записывает обращение и ответ в файл cuckoo.log в формате: ДАТА ВРЕМЯ NAME[PID] N
- При получении сигнала (на ваш выбор) скрипт записывает в файл cuckoo.log сообщение в формате: ДАТА ВРЕМЯ Shutdown! и завершает свою работу.

```

user1@eltex-practice2-v10:~/practice3$ cat > cuckoo.sh <<'EOF'
#!/bin/bash
# cuckoo.sh – сервер выдачи времени через именованный канал
DIR="$(cd "$(dirname "$0")" && pwd)"
RUN=/tmp/run
FIFO=$RUN/cuckoo.pid
LOG=$DIR/cuckoo.log

mkdir -p "$RUN"
[ -p "$FIFO" ] || mkfifo "$FIFO"
exec 3<>"$FIFO"

echo "$(date '+%F %T') Startup!" >> "$LOG"

shutdown() {
    echo "$(date '+%F %T') Shutdown!" >> "$LOG"
    exit 0
}
trap shutdown TERM INT

while read -r line <&3; do
    case "$line" in
        *"how much time do I have?")
            np="${line%:*}" # NAME[PID]
            pid="${np#*[]}"; pid="${pid%]}"
            N=$(( RANDOM % 9 + 2 )) # 2..10
            echo "$(date '+%F %T') $np $N" >> "$LOG"
            echo "$N" > "$RUN/reply.$pid"
            ;;
    esac
done
EOF
user1@eltex-practice2-v10:~/practice3$ chmod +x cuckoo.sh
user1@eltex-practice2-v10:~/practice3$ run bash -n cuckoo.sh && echo "синтаксис ОК"
синтаксис ОК
user1@eltex-practice2-v10:~/practice3$ █

```

Рис. 20 – Результат

2 Запустите скрипт cuckoo.sh при помощи nohup или screen

```

user1@eltex-practice2-v10:~/practice3$ run pgrep -af cuckoo.sh
39839 /bin/bash ./cuckoo.sh
39848 /bin/bash ./cuckoo.sh
user1@eltex-practice2-v10:~/practice3$ run ls -l /tmp/run/
total 0
prw-r----- 1 user1 user1 0 Aug 23 06:19 cuckoo.pid
user1@eltex-practice2-v10:~/practice3$ run cat cuckoo.log
2026-08-23 06:19:28 Startup!
2026-08-23 06:19:45 Startup!
user1@eltex-practice2-v10:~/practice3$ run pkill -f cuckoo.sh
user1@eltex-practice2-v10:~/practice3$ run pgrep -af cuckoo.sh
[1]- Done run nohup ./cuckoo.sh > /dev/null 2>&1
[2]+ Done run nohup ./cuckoo.sh > /dev/null 2>&1
user1@eltex-practice2-v10:~/practice3$ run pkill -f cuckoo.sh
user1@eltex-practice2-v10:~/practice3$ run pgrep -af cuckoo.sh
user1@eltex-practice2-v10:~/practice3$
user1@eltex-practice2-v10:~/practice3$ run tail -n 2 cuckoo.log
2026-08-23 06:34:36 Shutdown!
2026-08-23 06:34:36 Shutdown!
user1@eltex-practice2-v10:~/practice3$ run nohup ./cuckoo.sh >/dev/null 2>&1 &
sleep 1
run pgrep -af cuckoo.sh
run tail -n 1 cuckoo.log
[1] 39930
39932 /bin/bash ./cuckoo.sh
2026-08-23 06:35:46 Startup!
user1@eltex-practice2-v10:~/practice3$

```

Рис. 21 – Результат

```

user1@eltex-practice2-v10:~/practice3$ echo "proba[777]: how much time do I have?" > /tmp/run/cuckoo.pid
user1@eltex-practice2-v10:~/practice3$ run tail -n 5 cuckoo.log
2026-08-23 06:19:45 Startup!
2026-08-23 06:34:36 Shutdown!
2026-08-23 06:34:36 Shutdown!
2026-08-23 06:35:46 Startup!
2026-08-23 06:37:47 proba[777] 5
user1@eltex-practice2-v10:~/practice3$ run cat /tmp/run/reply.777
5
user1@eltex-practice2-v10:~/practice3$

```

Рис. 22 – Результат

3 Создайте скрипт на языке bash с именем `template_task.sh`, делающий следующее:

- При запуске проверяет, что имя скрипта не совпадает с `template_task.sh`, если совпадает - выходит с уведомлением «я бригадир, сам не работаю»
- При запуске дописывает в файл `report_имя_скрипта_без_полного_пути.log` в рабочем каталоге информацию: ДАТА ВРЕМЯ [PID] Скрипт запущен
- Запрашивает через именованный канал скрипта `cuckoo` время сообщением в указанном ранее формате и ждет такое количество секунд.

- Дописывает в файл report_имя_скрипта_без_полного_пути.log сообщение: ДАТА ВРЕМЯ [PID] Скрипт завершился, работал N секунд.

```
#!/bin/bash
```

```
# template_task.sh — «бригадир»: работает только под чужим именем
```

```
NAME="$(basename "$0")"
```

```
if [ "$NAME" = "template_task.sh" ]; then
```

```
    echo "я бригадир, сам не работаю"
```

```
    exit 1
```

```
fi
```

```
RUN=/tmp/run
```

```
FIFO=$RUN/cuckoo.pid
```

```
LOG="report_${NAME}.log"
```

```
echo "$(date '+%F %T') [$$] Скрипт запущен" >> "$LOG"
```

```
N=5
```

```
if [ -p "$FIFO" ]; then
```

```
    echo "${NAME}[$$]: how much time do I have?" > "$FIFO"
```

```
    for _ in $(seq 1 50); do
```

```
        [ -f "$RUN/reply.$$" ] && { N=$(cat "$RUN/reply.$$"); rm -f "$RUN/reply.$$"; break; }
```

```
        sleep 0.2
```

```
    done
```

```
fi
```

```
sleep "$N"
```

```
echo "$(date '+%F %T') [$$] Скрипт завершился, работал $N секунд." >> "$LOG"
```

```

user1@eltex-practice2-v10:~/practice3$ cat > template_task.sh <<'EOF'
#!/bin/bash
# template_task.sh – «бригадир»: работает только под чужим именем
NAME="$(basename "$0")"
if [ "$NAME" = "template_task.sh" ]; then
    echo "я бригадир, сам не работаю"
    exit 1
fi

RUN=/tmp/run
FIFO=$RUN/cuckoo.pid
LOG="report_${NAME}.log"

echo "$(date '+%F %T') [$$] Скрипт запущен" >> "$LOG"

N=5
if [ -p "$FIFO" ]; then
    echo "${NAME}[$$]: how much time do I have?" > "$FIFO"
    for _ in $(seq 1 50); do
        [ -f "$RUN/reply.$$" ] && { N=$(cat "$RUN/reply.$$"); rm -f "$RUN/reply.$$"; break; }
        sleep 0.2
    done
fi

sleep "$N"
echo "$(date '+%F %T') [$$] Скрипт завершился, работал $N секунд." >> "$LOG"
EOF
chmod +x template_task.sh
user1@eltex-practice2-v10:~/practice3$ run ./template_task.sh
я бригадир, сам не работаю

```

Рис. 23 – Результат

```

5
user1@eltex-practice2-v10:~/practice3$ run ln -sf template_task.sh worker1
user1@eltex-practice2-v10:~/practice3$ run ./worker1
user1@eltex-practice2-v10:~/practice3$ run cat report_worker1.log
2026-08-23 06:39:21 [39949] Скрипт запущен
2026-08-23 06:39:25 [39949] Скрипт завершился, работал 4 секунд.
user1@eltex-practice2-v10:~/practice3$ run tail -n 2 cuckoo.log
2026-08-23 06:37:47 proba[777] 5
2026-08-23 06:39:21 worker1[39949] 4
user1@eltex-practice2-v10:~/practice3$

```

Рис. 24 – Результат

4 Создайте скрипт на языке bash с именем `observer.sh`, читающий файл конфигурации со списком скриптов `observer.conf`, проверяющим их наличие в списке работающих процессов поиском в `/proc` и запускающих их в отключенном от терминала режиме (`nohup`) в случае отсутствия в нем. Информация о перезапуске дописывайте в файл `observer.log`

```
#!/bin/bash
```

```
# observer.sh — перезапускает задачи из observer.conf, если их нет в /proc
```

```
DIR="$(cd "$(dirname "$0")" && pwd)"
```

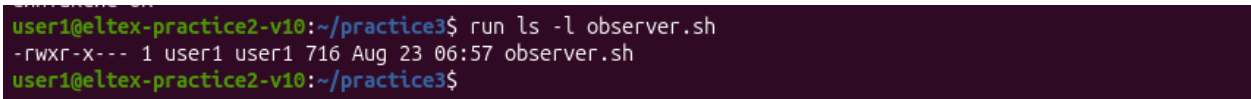


```

CONF="$DIR/observer.conf"
LOG="$DIR/observer.log"
cd "$DIR" || exit 1
[ -r "$CONF" ] || exit 1

while read -r task; do
    case "$task" in "|\\#*) continue ;; esac
    name="$(basename "$task")"
    found=0
    for c in /proc/[0-9]*/cmdline; do
        [ -r "$c" ] || continue
        if tr '\0' '\n' 2>/dev/null < "$c" | grep -q "/$name$"; then
            found=1; break
        fi
    done
    if [ "$found" -eq 0 ]; then
        echo "$(date '+%F %T') Перезапуск: $name (не найден в /proc)" >> "$LOG"
        nohup "$DIR/$name" >/dev/null 2>&1 &
    fi
done < "$CONF"

```



```

user1@eltex-practice2-v10:~/practice3$ run ls -l observer.sh
-rwxr-x--- 1 user1 user1 716 Aug 23 06:57 observer.sh
user1@eltex-practice2-v10:~/practice3$

```

Рис. 25 – Результат

5 Настройте запуск observer.sh посредством cron по расписанию – 1 раз в минуту

```

user1@eltex-practice2-v10:~/practice3$ run crontab -l
no crontab for user1
user1@eltex-practice2-v10:~/practice3$ ( crontab -l 2>/dev/null; echo "* * * * * /home/user1/practice3/observer.sh" ) | crontab -
user1@eltex-practice2-v10:~/practice3$ run crontab -l
* * * * * /home/user1/practice3/observer.sh
user1@eltex-practice2-v10:~/practice3$

```

Рис. 26 – Результат

6 Создайте несколько символьных ссылок на файл `template_task.sh` с различными именами (рабочие задачи), добавьте в файл конфигурации `observer.conf` соответствующие записи об этих задачах, включая исходный файл `template_task.sh`

```

user1@eltex-practice2-v10:~/practice3$ ls -l worker*
lrwxrwxrwx 1 user1 user1 16 Aug 23 06:39 worker1 -> template_task.sh
lrwxrwxrwx 1 user1 user1 16 Aug 23 06:40 worker2 -> template_task.sh
lrwxrwxrwx 1 user1 user1 16 Aug 23 06:40 worker3 -> template_task.sh
user1@eltex-practice2-v10:~/practice3$

```

Рис. 27 – Результат

```

user1@eltex-practice2-v10:~/practice3$ cat > observer.conf <<'EOF'
template_task.sh
worker1
worker2
worker3
EOF
run cat observer.conf
template_task.sh
worker1
worker2
worker3
user1@eltex-practice2-v10:~/practice3$

```

Рис. 28 – Результат

7 Соберите статистику работы в виде набора файлов `report_*.log`, `observer.log`, `cuckoo.log` приложите их вместе с исходными текстами скриптов в качестве отчета в виде сжатого архива `tar`. Не забудьте остановить процесс, удалив задачу в `cron`!

```

user1@eltex-practice2-v10:~/practice3$ run crontab -r
user1@eltex-practice2-v10:~/practice3$ run crontab -l
no crontab for user1
user1@eltex-practice2-v10:~/practice3$ kill -f observer.sh
user1@eltex-practice2-v10:~/practice3$ kill -f "practice3/worker"
user1@eltex-practice2-v10:~/practice3$ pgrep -af "practice3/worker"
user1@eltex-practice2-v10:~/practice3$ kill -TERM $(pgrep -f cuckoo.sh)
user1@eltex-practice2-v10:~/practice3$ tail -n 2 cuckoo.log
2026-08-26 03:03:01 worker3[4185845] 9
2026-08-26 03:05:03 Shutdown!
[1]+  Done                  run nohup ./cuckoo.sh > /dev/null 2>&1
user1@eltex-practice2-v10:~/practice3$ pgrep -af cuckoo.sh
user1@eltex-practice2-v10:~/practice3$

```

Рис. 29 – Результат

```
user1@eltex-practice2-v10:~/practice3$ tar -czf task3_part2_report.tar.gz \
    cuckoo.sh template_task.sh observer.sh observer.conf \
    cuckoo.log observer.log report_*.log
user1@eltex-practice2-v10:~/practice3$ ls -lh task3_part2_report.tar.gz
-rw-r----- 1 user1 user1 313K Aug 26 03:06 task3_part2_report.tar.gz
user1@eltex-practice2-v10:~/practice3$ tar -tzf task3_part2_report.tar.gz
cuckoo.sh
template_task.sh
observer.sh
observer.conf
cuckoo.log
observer.log
report_worker1.log
report_worker2.log
report_worker3.log
user1@eltex-practice2-v10:~/practice3$
```

Рис. 30 – Результат